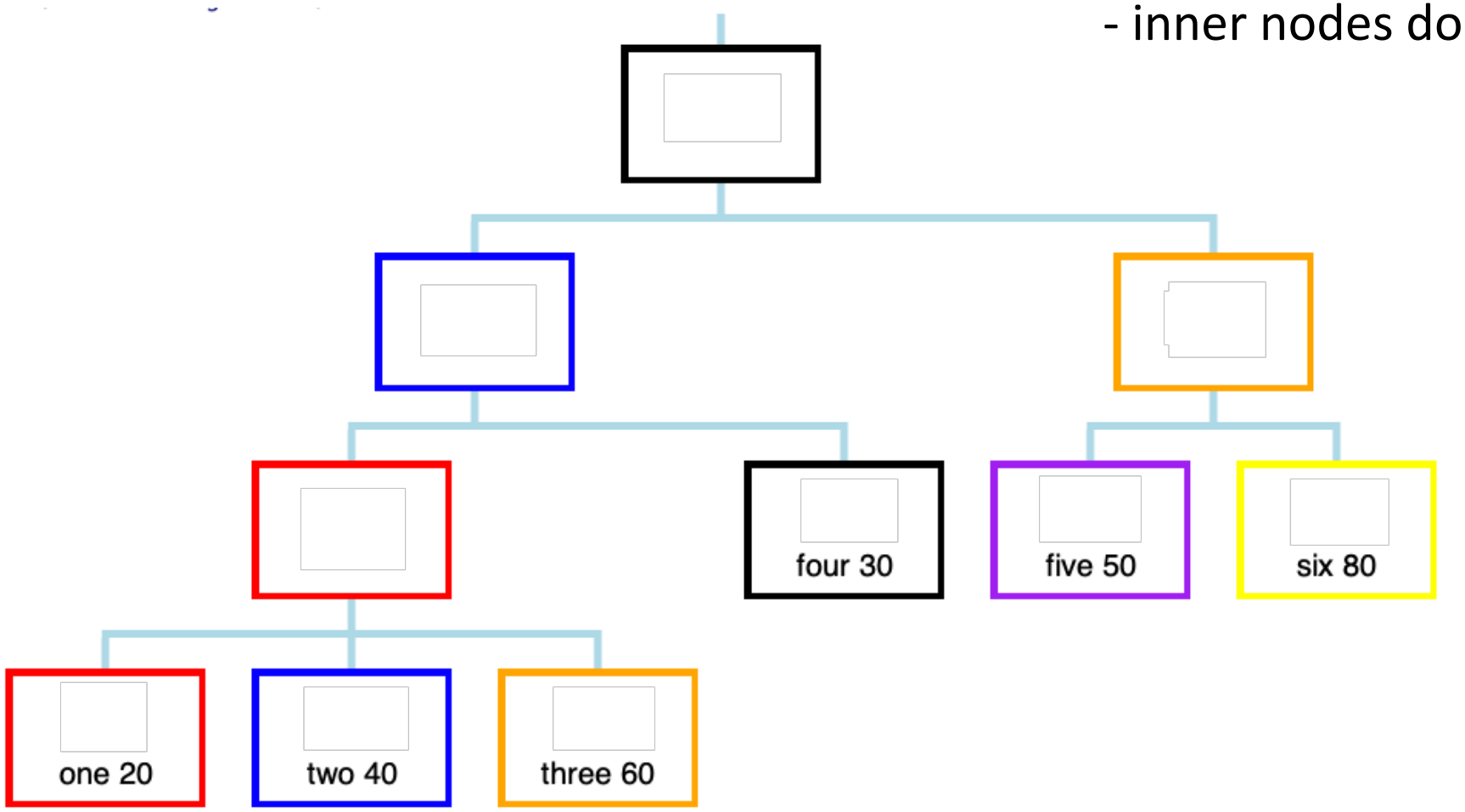
TODAY

- Another day of reaping the rewards of the work you have done so far

- Two important new ideas
  - mutual reference / recursion
    - The hard parts won't actually be very hard
      - just trusting natural recursions as usual
      - not blind trust - we have to get base, contribution and combination right!
    - The easy parts will require learning some new details
    - minor housekeeping changes with @htdf @htdd organization
  - Functions that can fail
    - or false in signatures
    - Try-catch template

Some tree terminology:
- leaf nodes have no subs
- inner nodes do have subs



```
(@htdd Region ListOfRegion)

(define-struct leaf (label weight color))
(define-struct inner (color subs))

;; Region is one of:
;;  - (make-leaf String Natural Color)
;;  - (make-inner Color ListOfRegion)

;; ListOfRegion is one of:
;;  - empty
;;  - (cons Region ListOfRegion)
;; interp. a list of regions
```

```
(@template-origin Region)

(define (fn-for-region r)
  (cond [(leaf? r)
         (... (leaf-label r)
              (leaf-weight r)
              (leaf-color r))]
        [else
         (... (inner-color r)
              (fn-for-lor (inner-subs r)))]))

(@template-origin ListOfRegion)

(define (fn-for-lor lor)
  (cond [(empty? lor) (...)]
        [else
         (... (fn-for-region (first lor))
              (fn-for-lor (rest lor)))]))
```

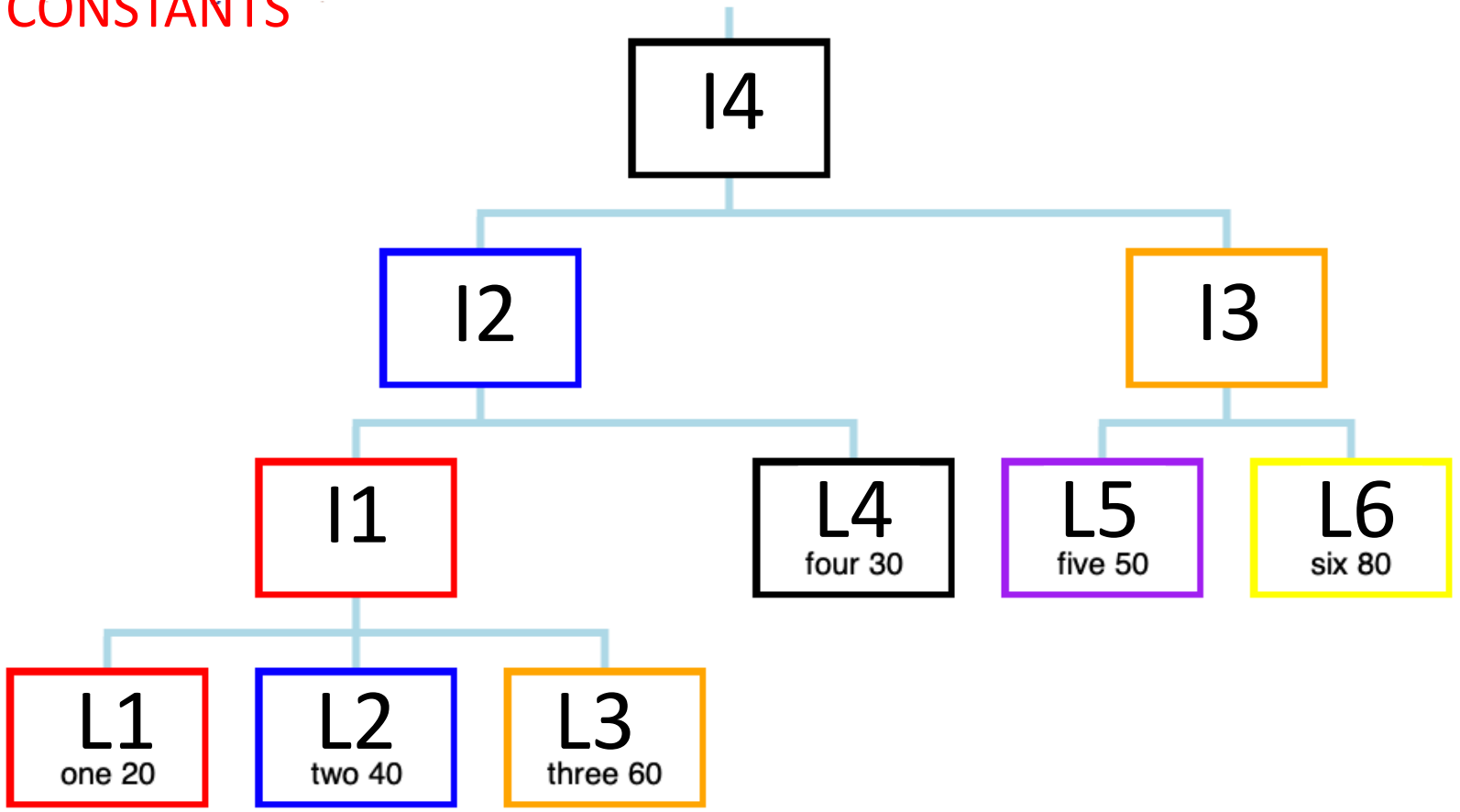
NMR

NMR

NR

mutual recursion means
these functions come in pairs
(2 or more in general)

L1, L2, I4 etc. are the
NAMES OF THE CONSTANTS



```
(define (total-weight--region r)
  (cond [(leaf? r)
         (... (leaf-label r)
              (leaf-weight r)
              (leaf-color r))]
        [else
         (... (inner-color r)
              (total-weight--lor (group-subs r)))]))
```

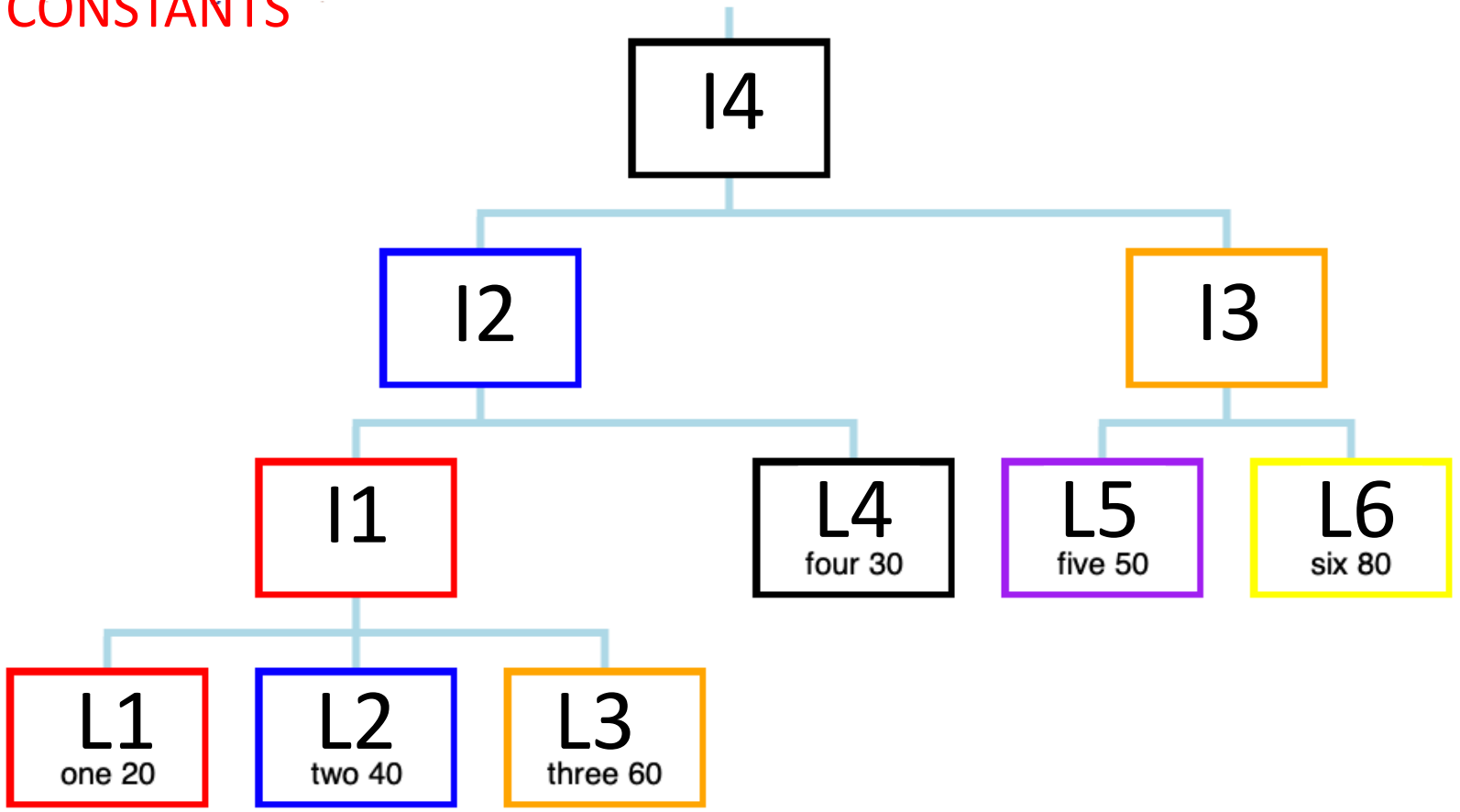
result will be?

```
(define (total-weight--lor lor)
  (cond [(empty? lor) (...)]
        [else
         (... (total-weight--region (first lor))
              (total-weight--lor (rest lor)))]))
```

result will be?

result will be?

L1, L2, I4 etc. are the
NAMES OF THE CONSTANTS



```
(define (all-with-color--region r)
  (cond [(leaf? r)
         (... (leaf-label r)
              (leaf-weight r)
              (leaf-color r))]
        [else
         (... (inner-color r)
              (all-with-color--lor (group-subs r)))]))
```

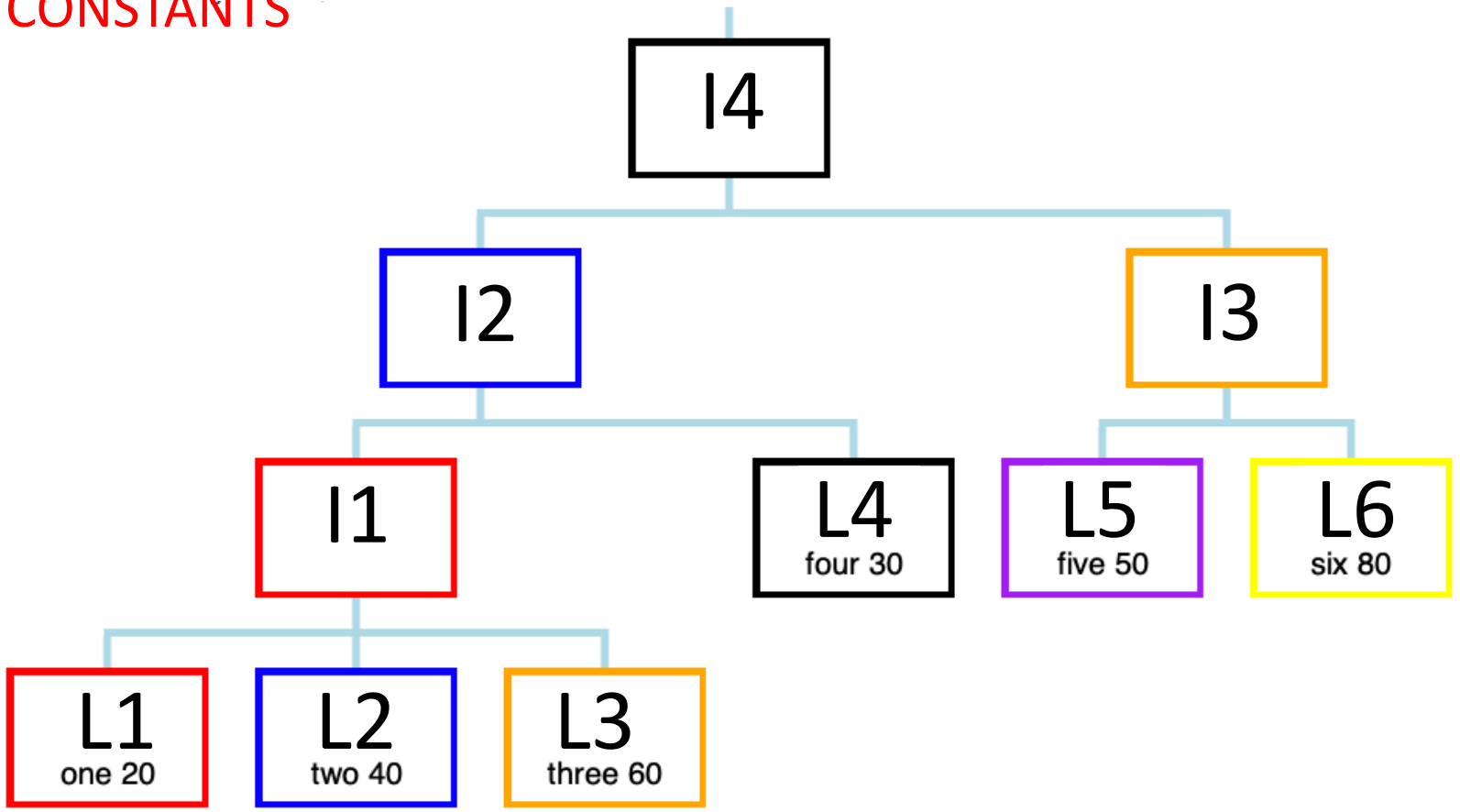
result will be?

```
(define (all-with-color--lor lor)
  (cond [(empty? lor) (...)]
        [else
         (... (all-with-color--region (first lor))
              (all-with-color--lor (rest lor)))]))
```

result will be?

result will be?

L1, L2, I4 etc. are the
NAMES OF THE CONSTANTS



```
(define (find-region--region r)
  (cond [(leaf? r)
         (... (leaf-label r)
              (leaf-weight r)
              (leaf-color r))]
        [else
         (... (inner-color r)
              (find-region--lor (group-subs r)))]))
```

result will be?

```
(define (find-region--lor lor)
  (cond [(empty? lor) (...)]
        [else
         (... (find-region--region (first lor))
              (find-region--lor (rest lor)))]))
```

result will be?

result will be?